

Relatório Técnico

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

1. Introdução

1.1 Contexto da atividade

Este projeto foi desenvolvido como parte da atividade Agentes para CSV, cujo objetivo é construir uma solução capaz de permitir a consulta de dados estruturados por meio de linguagem natural.

A proposta consiste em facilitar a exploração de arquivos CSV por usuários que não necessariamente possuem conhecimentos em programação, SQL ou ferramentas tradicionais de análise de dados.

Como domínio de aplicação, foram utilizados conjuntos de dados relacionados a Notas Fiscais Eletrônicas (NF-e). A aplicação permite que o usuário envie um arquivo compactado contendo os dados, realize o processamento automático das tabelas e, posteriormente, interaja com um agente inteligente por meio de perguntas em linguagem natural.

Além da consulta conversacional, foi desenvolvido um painel analítico contendo indicadores e visualizações gráficas calculados diretamente a partir dos mesmos DataFrames utilizados pelo agente.

O protótipo foi construído utilizando Python, Streamlit, Pandas, Plotly, LangChain e um modelo da família Gemini, acessado por meio da integração `langchain-google-genai`.

1.2 Objetivo do projeto

O objetivo principal do projeto é desenvolver uma interface inteligente para análise de dados fiscais que permita:

- realizar o upload de um arquivo ZIP contendo um ou mais arquivos CSV;
- carregar e transformar automaticamente os dados;
- interpretar um dicionário de dados fornecido com o conjunto;
- criar automaticamente um dicionário simplificado quando este não estiver disponível;
- remover registros duplicados;
- identificar semanticamente as principais tabelas do conjunto de dados;
- disponibilizar indicadores e gráficos exploratórios;
- permitir consultas em linguagem natural;
- utilizar um agente inteligente para decidir como consultar os DataFrames;
- executar operações com Pandas para obter as respostas;
- apresentar os resultados ao usuário em linguagem natural.

O foco da solução está na transformação de dados estruturados em informações acessíveis por meio da interação com um agente de inteligência artificial.

1.3 Escopo da solução

A solução foi desenvolvida com caráter educacional. O objetivo não é oferecer uma plataforma comercial pronta para produção, mas demonstrar a integração entre agentes inteligentes, modelos de linguagem e dados tabulares.

Embora o requisito principal da atividade seja trabalhar com arquivos CSV, a implementação também passou a oferecer suporte adicional a arquivos XML ampliando as possibilidades de entrada de dados.

2. Tecnologias e arquitetura da solução

2.1 Tecnologias utilizadas

As principais tecnologias utilizadas no desenvolvimento foram:

Tecnologia	Finalidade
Python	Linguagem principal do projeto
Streamlit	Desenvolvimento da interface web
Pandas	Manipulação e análise dos dados
LangChain	Framework de desenvolvimento do agente
LangChain Experimental	Criação do Pandas DataFrame Agent
Google Gemini	Modelo de linguagem utilizado pelo agente
<code>langchain-google-genai</code>	Integração entre LangChain e Gemini
Plotly	Construção dos gráficos interativos
<code>python-dotenv</code>	Carregamento da chave da API
<code>zipfile</code>	Leitura e validação de arquivos compactados
<code>xml.etree.ElementTree</code>	Leitura adicional de arquivos XML de NF-e

O framework de agentes escolhido foi o LangChain. O agente é criado por meio de `create_pandas_dataframe_agent`, permitindo que o modelo de linguagem utilize operações Pandas sobre os DataFrames carregados.

2.2 Estrutura do projeto

A aplicação foi organizada de maneira modular:

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

```
.
├─ app2.py
├─ .streamlit/
│   └─ config.toml
├─ src/
│   ├── agente.py
│   ├── carregador.py
│   ├── kpis.py
│   ├── validador.py
│   └─ xml_carregador.py
└─ requirements.txt
```

Essa organização separa a interface, o processamento dos dados, o agente, os cálculos analíticos e os mecanismos de validação.

app.py

Responsável pela:

- interface Streamlit;
- realização do upload;
- controle da sessão;
- apresentação dos KPIs;
- apresentação dos gráficos;
- interface de chat;
- integração dos demais componentes.

src/agente.py

Responsável pela:

- configuração do modelo Gemini;
- criação do Pandas DataFrame Agent;
- construção das instruções do agente;
- envio do dicionário de dados ao modelo;
- associação entre DataFrames e arquivos;
- execução das perguntas;
- inclusão de histórico recente;
- tratamento da resposta retornada.

src/carregador.py

Responsável pela:

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

- leitura dos CSVs;
- reconhecimento de diferentes codificações;
- identificação automática de separadores;
- transformação de tipos;
- conversão de datas;
- conversão de valores monetários;
- conversão de quantidades;
- criação de variáveis auxiliares;
- remoção de duplicatas;
- carregamento do dicionário de dados.

src/kpis.py

Responsável pela:

- identificação das tabelas de notas e itens;
- localização semântica das colunas;
- cálculo dos KPIs;
- formatação dos valores;
- geração das visualizações Plotly.

src/validador.py

Responsável pela extração segura dos arquivos contidos no ZIP.

src/xml_carregador.py

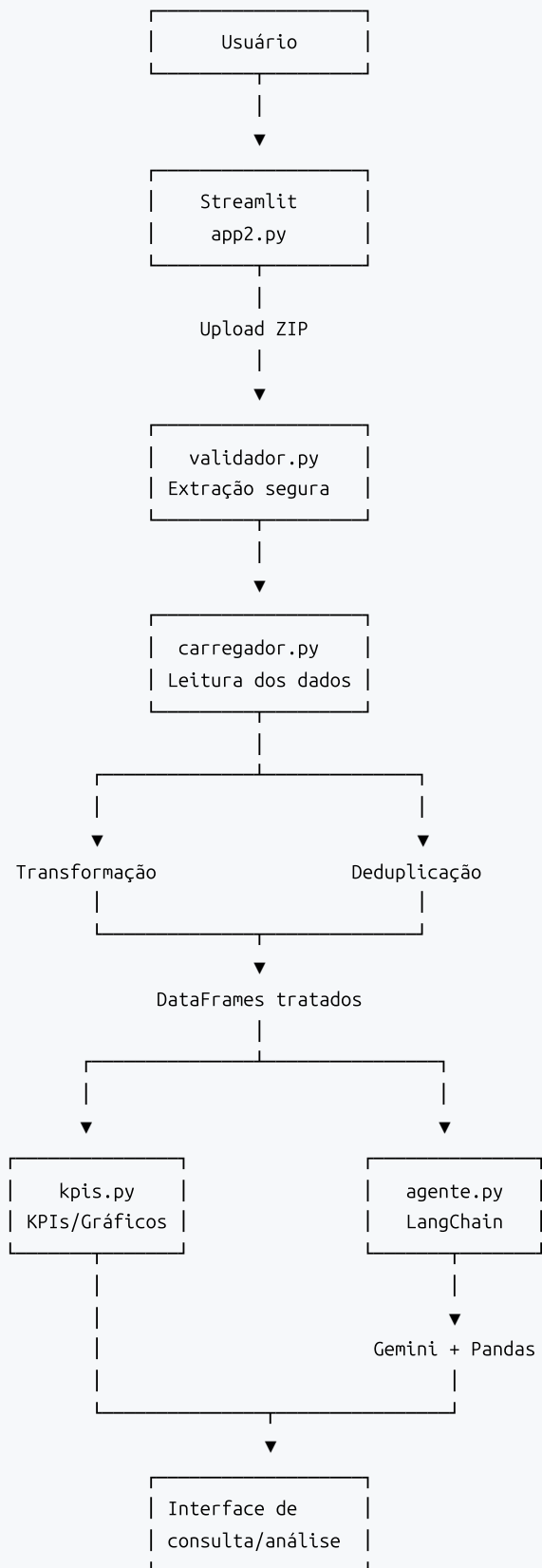
Responsável pelo suporte adicional à leitura de XMLs de NF-e(não validado).

A divisão modular corresponde à estrutura efetivamente implementada no projeto.

2.3 Arquitetura geral

A arquitetura da solução pode ser representada pelo seguinte fluxo:

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV



Essa separação reduz o acoplamento entre os componentes e facilita manutenção, testes e evolução do projeto.

3. Processamento e preparação dos dados

3.1 Upload e validação do arquivo ZIP

A primeira etapa ocorre por meio da interface Streamlit, onde o usuário envia um arquivo no formato `.ZIP`.

O componente utilizado é:

```
st.sidebar.file_uploader(  
    "Envie um arquivo ZIP",  
    type=["zip"]  
)
```

Antes do processamento, são realizadas verificações sobre o arquivo recebido.

O módulo `validador.py` implementa medidas que incluem:

- limite da quantidade de arquivos no ZIP;
- limite total do conteúdo descompactado;
- verificação dos caminhos de destino;
- proteção contra tentativa de extração para fora da pasta temporária.

Esse procedimento reduz riscos associados à extração indiscriminada de arquivos compactados.

3.2 Leitura dos arquivos CSV

Após a extração, o sistema procura recursivamente pelos arquivos CSV disponíveis.

A função de leitura tenta diferentes codificações:

```
UTF-8  
UTF-8 com BOM  
Latin-1
```

Também é utilizada detecção automática do separador, permitindo trabalhar com arquivos separados, por exemplo, por vírgula ou ponto e vírgula.

Essa estratégia aumenta a compatibilidade com arquivos provenientes de diferentes sistemas.

3.3 Conversão e transformação dos dados

Após a leitura, os DataFrames passam por um processo automático de transformação.

Entre as operações realizadas estão:

- identificação de colunas de data;
- conversão para `datetime`;

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

- conversão de valores monetários;
- conversão de quantidades;
- conversão de identificadores numéricos;
- criação de variáveis auxiliares relacionadas ao tempo;
- criação ou padronização do valor total dos itens.

Quando existe uma coluna de data de emissão válida, são criadas variáveis auxiliares:

```
dia  
hora  
dia_semana
```

A lógica utilizada é do tipo `best effort`. Caso determinada coluna esperada não exista, a transformação correspondente é ignorada sem interromper o processamento completo.

3.4 Remoção de registros duplicados

A remoção dos registros duplicados ocorre após a etapa de transformação.

A ordem adotada foi:

```
leitura  
↓  
transformação  
↓  
deduplicação
```

Essa decisão é importante porque valores textualmente diferentes podem representar o mesmo valor depois da conversão.

Por exemplo:

```
1.234,56  
1234,56
```

Após a transformação numérica, registros que anteriormente pareciam diferentes podem ser identificados corretamente como equivalentes.

3.5 Dicionário de dados

O dicionário de dados fornece informações adicionais sobre o significado das colunas.

Quando o arquivo `dicionario_dados.csv` é fornecido, a aplicação verifica a existência dos seguintes campos:

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

```
arquivo
coluna
descricao
tipo
```

Caso o dicionário não seja encontrado, a aplicação constrói automaticamente uma versão simplificada utilizando os nomes das tabelas, colunas e respectivos tipos.

Posteriormente, esse dicionário é convertido para texto e incluído nas instruções enviadas ao agente.

O carregador implementa tanto a validação do dicionário existente quanto sua criação automática quando necessário.

3.6 Identificação das tabelas

Como os nomes dos arquivos podem variar entre diferentes conjuntos de dados, a aplicação utiliza as características das colunas para identificar as tabelas.

O mecanismo atribui pontuações com base nos campos existentes e tenta determinar qual DataFrame representa:

- cabeçalho das notas fiscais;
- itens das notas fiscais.

Para a tabela de notas são considerados campos como:

```
chave_de_acesso
numero
serie
data_emissao
valor_nota_fiscal
cpf/cnpj_emitente
razao_social_emitente
uf_emitente
uf_destinatario
```

Para a tabela de itens são considerados, entre outros:

```
numero_produto
descricao_do_produto/servico
quantidade
unidade
valor_unitario
valor_total_item
codigo_ncm/sh
cfop
```

Dessa forma, a identificação é feita prioritariamente pela estrutura semântica das tabelas, em vez de depender exclusivamente do nome do arquivo.

3.7 Suporte adicional a XML ()

Apesar de o requisito principal da atividade estar relacionado a arquivos CSV, foi implementado suporte adicional a XML de NF-e.

O módulo `xml_carregador.py` :

- procura arquivos XML;
- interpreta a estrutura de NF-e;
- extrai dados principais da nota;
- extrai os produtos;
- cria DataFrames separados para notas e itens;
- registra erros encontrados durante a leitura.

Entre as informações extraídas dos itens estão:

- código do produto;
- descrição;
- NCM;
- CFOP;
- unidade;
- quantidade;
- valor unitário;
- valor do produto.

Essa funcionalidade deve ser entendida como uma extensão experimental do MVP e não como um requisito necessário da atividade. Até o momento, o módulo de leitura de XML não passou por uma etapa formal de testes e validação com diferentes conjuntos de arquivos NF-e. Por esse motivo, sua implementação deve ser considerada preliminar

4. Agente inteligente

4.1 Framework escolhido

O núcleo da solução utiliza o LangChain como framework para criação e orquestração do agente.

A opção pelo LangChain permitiu utilizar um agente especializado em DataFrames Pandas, evitando a necessidade de converter os dados para um banco SQL apenas para realizar as consultas.

O agente recebe diretamente os DataFrames carregados e pode decidir quais operações Pandas executar para responder à solicitação do usuário.

4.2 Modelo de linguagem

O projeto possui uma lista de modelos Gemini configurados, sendo utilizado na versão analisada:

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

```
gemini-3.1-flash-lite
```

A configuração principal utiliza:

```
temperature=0  
thinking_level="Low"
```

A temperatura igual a zero busca reduzir variações desnecessárias em respostas quantitativas, enquanto o nível de raciocínio baixo foi utilizado para equilibrar capacidade de decisão e tempo de resposta.

4.3 Pandas DataFrame Agent

O processo realizado pelo agente pode ser representado como:

```
Pergunta em linguagem natural  
  ↓  
  LLM  
  ↓  
Interpretação da solicitação  
  ↓  
Seleção do DataFrame  
  ↓  
Seleção das colunas  
  ↓  
Determinação da operação  
  ↓  
Execução com Pandas  
  ↓  
Resultado  
  ↓  
Resposta em linguagem natural
```

Os DataFrames são fornecidos diretamente ao Pandas DataFrame Agent, permitindo ao modelo solicitar a execução de operações sobre os dados.

4.4 Contexto fornecido ao agente

Além dos DataFrames, é fornecido ao agente um mapeamento contendo informações sobre cada tabela.

O contexto assume uma estrutura semelhante a:

```
df1: tabela 'nome_da_tabela', X linhas, colunas [...]  
df2: tabela 'outra_tabela', Y linhas, colunas [...]
```

Também é incluído o conteúdo textual do dicionário de dados.

Essa estratégia fornece ao modelo informações adicionais para decidir qual DataFrame e quais colunas são apropriados para cada consulta.

4.5 Estratégia de interpretação das perguntas

Durante o desenvolvimento foram identificados problemas associados principalmente à escolha incorreta de tabelas, confusão entre colunas semelhantes e utilização inadequada da granularidade.

Por esse motivo, o prompt foi organizado para orientar o seguinte fluxo:

```
Identificar a entidade
    ↓
Identificar a métrica
    ↓
Selecionar o DataFrame
    ↓
Verificar a granularidade
    ↓
Selecionar as colunas
    ↓
Escolher a operação matemática
    ↓
Executar com Pandas
    ↓
Validar o resultado
    ↓
Formatar a resposta
```

As instruções também orientam o agente a:

- utilizar somente os dados disponíveis;
- não inventar valores ou colunas;
- utilizar identificadores únicos quando necessário;
- diferenciar tabelas de notas e itens;
- evitar calcular totais a partir de amostras ou `.head()` ;
- utilizar valores numéricos completos durante os cálculos;
- arredondar somente na apresentação final.

Essas regras fazem parte do prompt atualmente utilizado pelo agente.

4.6 Granularidade das tabelas

Um dos principais pontos considerados durante o desenvolvimento foi a diferença entre a granularidade das tabelas.

Tabela de notas

Cada registro representa um documento fiscal. Entre as informações possíveis estão:

- chave de acesso;
- número;

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

- data de emissão;
- valor da nota;
- emitente;
- destinatário;
- UF.

Tabela de itens

Cada registro representa um produto ou serviço relacionado a uma nota fiscal.

Entre as informações disponíveis estão:

- descrição do produto;
- quantidade;
- valor unitário;
- valor total do item;
- NCM;
- CFOP.

Essa distinção é necessária porque uma mesma nota pode possuir vários itens. Portanto, operações sobre a tabela de itens não devem ser interpretadas automaticamente como operações sobre a quantidade de documentos fiscais.

4.7 Histórico da conversa

A aplicação também utiliza parte do histórico recente da interação. Na implementação atual são consideradas até 6 mensagens anteriores. Essa estratégia permite perguntas de continuidade ao mesmo tempo em que evita o crescimento indefinido do contexto enviado ao modelo.

5. Interface e recursos analíticos

5.1 Interface de carga

A Interface A é a barra lateral do Streamlit.

O usuário:

1. seleciona o arquivo ZIP;
2. visualiza os arquivos encontrados;
3. solicita o processamento;
4. recebe mensagens de sucesso ou erro.

Quando o processamento é concluído, os DataFrames, o dicionário e o agente são armazenados no estado da sessão, permitindo a disponibilização automática da área de análise.

5.2 Interface de consulta

Após o processamento, o usuário passa a ter acesso à Interface B. O chat permite realizar perguntas em linguagem natural, como:

Qual produto apresentou maior volume comercializado?

ou:

Qual foi o valor total das notas fiscais?

Durante a execução, a interface apresenta uma indicação de processamento e, ao final, exibe a resposta do agente em linguagem natural.

Também são tratadas exceções para evitar que uma falha de consulta interrompa toda a aplicação.

5.3 Indicadores

O dashboard apresenta quatro indicadores principais.

Indicador	Significado
Total de notas fiscais	Quantidade de documentos fiscais distintos
Valor total	Soma do valor das notas
Itens processados	Número de registros da tabela de itens após o processamento
Ticket médio	Valor total dividido pela quantidade de notas

A aplicação também calcula internamente a quantidade comercializada, correspondente à soma das quantidades presentes nos itens.

Para determinar a quantidade de notas fiscais, a aplicação utiliza preferencialmente a quantidade de valores distintos da coluna `chave_de_acesso`.

5.4 Visualizações gráficas

Além dos KPIs, foram desenvolvidas diferentes visualizações com Plotly.

5.4.1 Distribuição por hora

Apresenta a quantidade de notas emitidas em cada hora do dia.

O eixo horizontal representa:

Hora do dia

e o eixo vertical:

Quantidade de notas

5.4.2 Análise geográfica

São disponibilizadas visualizações para:

- Top 10 UF's dos emitentes;
- Top 10 UF's dos destinatários;
- valor total das notas por UF emitente.

5.4.3 Ranking de produtos

A aplicação possui rankings considerando:

- valor total dos produtos;
- quantidade comercializada.

5.4.4 Análise temporal

Também é apresentada a evolução da quantidade comercializada por data de emissão.

O cálculo utiliza a soma da quantidade comercializada dos itens em cada dia.

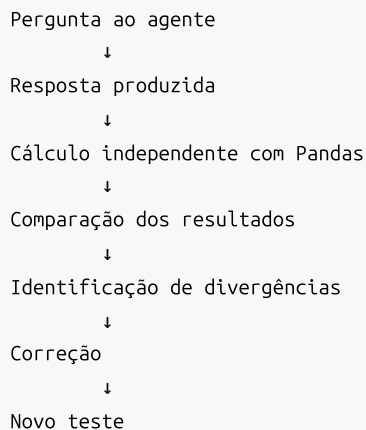
O dashboard reúne indicadores, análise horária, geográfica, de produtos e temporal em uma mesma interface.

6. Validação e testes da solução

6.1 Estratégia de validação

Durante o desenvolvimento foi adotada uma abordagem de validação independente das respostas produzidas pelo agente.

O procedimento utilizado pode ser resumido da seguinte forma:



O objetivo foi evitar considerar uma resposta correta apenas porque o texto produzido pelo modelo parecia plausível.

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

Essa metodologia possibilitou identificar problemas reais e aperfeiçoar tanto as funções de processamento quanto as instruções enviadas ao agente.

6.2 Perguntas realizadas

Foram utilizadas diferentes perguntas para verificar a consistência das respostas.

Pergunta 1:

Quantas notas fiscais existem nos dados?

Resposta validada:

143.797 notas fiscais

A referência foi calculada diretamente com Pandas:

```
df_notas["chave_de_acesso"].nunique()
```

Pergunta 2 :

Qual o valor total das notas fiscais?

Resposta validada:

R\$ 7.055.414.321,82

Cálculo de referência:

```
df_notas["valor_nota_fiscal"].sum()
```

Pergunta 3:

Qual o ticket médio das notas fiscais?

Valor de referência:

R\$ 49.065,10

Cálculo:

```
valor_total = df_notas["valor_nota_fiscal"].sum()
total_notas = df_notas["chave_de_acesso"].nunique()

ticket_medio = valor_total / total_notas
```

Esse teste foi importante porque demonstrou que cálculos derivados dependem da correta escolha do identificador e da granularidade utilizada.

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

Pergunta 4:

Qual produto apresentou a maior quantidade comercializada?

Resultado:

DOLUTEGRAVIR SOD 50MG CX50CT FR30CPR FAR

Cálculo de referência:

```
(
    df_itens
    .groupby("descrição_do_produto/serviço")["quantidade"]
    .sum()
    .sort_values(ascending=False)
    .head(1)
)
```

Quantidade agregada:

71.503.270

Pergunta 5:

Qual produto apresentou o maior valor total?

Resultado:

BULK INTERMEDIARIO; PRODUTO 0; SURECHECK; HIV; CHEMBIO

Cálculo de referência:

```
(
    df_itens
    .groupby("descrição_do_produto/serviço")["valor_total_item"]
    .sum()
    .sort_values(ascending=False)
    .head(1)
)
```

Valor agregado:

R\$ 933.958.190,60

As perguntas e os cálculos de referência utilizados na validação estão registrados no relatório original de desenvolvimento.

6.3 Problemas identificados durante os testes

O processo de validação permitiu identificar e corrigir situações como:

- contagem do número da nota em vez da chave de acesso;
- confusão entre quantidade comercializada e quantidade de registros;
- seleção incorreta da coluna de hora;
- escolha ambígua entre diferentes colunas de valor;
- mistura entre métricas da tabela de itens e métricas da tabela de notas.

Esses problemas demonstraram que a qualidade do agente depende não apenas do modelo de linguagem utilizado, mas também da estrutura dos dados organizadas e das instruções fornecidas.

7. Boas práticas, segurança e limitações

7.1 Organização modular

Uma das decisões de projeto foi manter responsabilidades separadas.

A solução foi dividida em componentes relacionados a:

- interface;
- agente;
- carregamento;
- transformação;
- validação;
- indicadores e gráficos;
- leitura de XML.

Essa organização facilita alterações individuais sem exigir modificações em toda a aplicação.

7.2 Utilização de Pandas como ferramenta

Como os dados já são carregados em DataFrames, Pandas foi utilizado como principal ferramenta analítica.

O agente pode executar operações como:

```
sum  
mean  
median  
nunique  
groupby  
filtros  
ordenações  
rankings  
percentuais  
agregações temporais
```

Essa escolha evita introduzir uma camada adicional de banco de dados sem necessidade para o escopo do projeto.

7.3 Tratamento de erros

A aplicação contempla diferentes cenários de falha, incluindo:

- arquivo ZIP inválido;
- ZIP excessivamente grande;
- arquivo CSV vazio;
- ausência de CSV ou XML válido;
- arquivo XML inválido;
- ausência de colunas necessárias para determinados gráficos;
- falha durante uma pergunta;
- ausência da chave da API.

Quando uma análise específica não pode ser construída, a interface procura informar o problema sem interromper toda a aplicação.

7.4 Proteção de credenciais

A chave de acesso ao modelo não é armazenada diretamente no código.

A aplicação utiliza a variável:

```
GOOGLE_API_KEY
```

carregada por meio de `python-dotenv`.

Caso a variável não seja encontrada, o usuário recebe um aviso.

Essa estratégia evita incluir diretamente a credencial nos arquivos Python versionados.

7.5 Segurança na execução

O Pandas DataFrame Agent necessita executar código Python para realizar operações sobre os DataFrames.

Agente Inteligente para Consulta de Dados Fiscais em Arquivos CSV

Por esse motivo, a criação do agente utiliza:

```
allow_dangerous_code=True
```

Essa característica deve ser considerada dentro do contexto do projeto.

A solução foi desenvolvida como MVP educacional para execução em ambiente controlado e não como aplicação preparada para exposição pública irrestrita.

7.6 Limitações

A solução apresenta algumas limitações associadas ao seu caráter de protótipo.

Interpretação do modelo de linguagem

Perguntas muito ambíguas podem ser interpretadas de maneiras diferentes pelo agente.

Estrutura dos dados: A identificação automática das tabelas funciona melhor quando o esquema possui características compatíveis com os conjuntos de dados fiscais utilizados durante o desenvolvimento.

Latência

Perguntas mais complexas podem exigir várias interações entre o modelo de linguagem e as ferramentas Pandas.

Uso de memória

Os DataFrames são mantidos em memória, estabelecendo limites práticos para o volume dos arquivos processados.

Execução dinâmica

Como o agente executa código para analisar os DataFrames, seu uso é mais apropriado em ambiente controlado.

8. Uso de ferramentas de inteligência artificial no desenvolvimento

Durante o desenvolvimento da atividade, foram utilizadas ferramentas baseadas em modelos de linguagem como apoio ao processo de programação, revisão técnica e elaboração da documentação.

Entre as ferramentas utilizadas estiveram modelos da Anthropic Claude, OpenAI, DeepSeek e Google Gemini.

O uso dessas ferramentas teve caráter assistivo, incluindo atividades como:

- apoio na organização e revisão do código;
- identificação de possíveis erros e inconsistências;
- sugestões de melhorias em funções e fluxos de processamento;
- apoio na elaboração e revisão de prompts;
- auxílio na validação lógica de cálculos e consultas;
- organização e revisão da documentação técnica.

As decisões finais sobre arquitetura, funcionamento da aplicação, tratamento dos dados, validação das respostas e integração entre os diferentes componentes foram definidas e verificadas ao longo do desenvolvimento do projeto. Da mesma forma, este relatório técnico foi elaborado com apoio de modelos de linguagem, utilizados principalmente para revisão textual e estruturação das seções.

9. Conclusão

O projeto desenvolvido demonstra a utilização de agentes inteligentes como intermediários entre usuários e dados estruturados.

A aplicação permite que arquivos contendo informações fiscais sejam enviados, processados e posteriormente consultados utilizando linguagem natural, reduzindo a necessidade de o usuário conhecer previamente Pandas, SQL ou outras ferramentas de análise.

O LangChain foi utilizado como camada de orquestração entre o modelo Gemini e os DataFrames Pandas. A partir da pergunta realizada, o agente interpreta a solicitação, identifica os dados relevantes, determina uma operação analítica e utiliza Pandas para produzir o resultado.

Durante o desenvolvimento verificou-se que o principal desafio não está apenas em conectar um modelo de linguagem a arquivos CSV. Para produzir resultados consistentes, também é necessário considerar corretamente:

- a tabela utilizada;
- a granularidade dos dados;
- o identificador de cada entidade;
- as colunas envolvidas;
- a operação matemática correspondente à pergunta.

Por esse motivo, a estratégia de desenvolvimento incluiu não apenas a construção do agente, mas também a validação independente das respostas por meio de cálculos Pandas. Esse processo permitiu identificar ambiguidades e corrigir comportamentos que poderiam gerar resultados quantitativamente incorretos.

A solução também incorporou recursos complementares ao requisito básico da atividade, como dashboard analítico, indicadores, gráficos interativos, transformação automática dos tipos de dados, remoção de duplicatas, geração de dicionário, identificação semântica das tabelas, validação segura do ZIP e suporte adicional a XML de NF-e.

O resultado é um MVP funcional e modular, capaz de demonstrar de maneira prática a integração entre agentes inteligentes, modelos de linguagem e conjuntos de dados estruturados.